SafeContract: Composable Action-Space Monitors for VLA Deployment

Krishnam Gupta
Independent Research
San Francisco, CA
kayjee1994@gmail.com

Abstract—Vision-Language-Action (VLA) models predict robot actions from images and instructions, but no standardized action-space monitoring exists for their deployment. We introduce SafeContract, a design-by-contract [5] monitoring toolkit that wraps any VLA policy with composable action-space contracts via a Python decorator. We show that the monitoring signal from a lightweight action-space contract reveals task-specific behavioral patterns and detects distribution shift. Bounds enforcement is a byproduct; the primary value is observability. Controlled ablations demonstrate zero false positives on clean policies while catching all violations on drifting and jerky policies. Across 130 LIBERO tasks (40 standard + 90 diverse), each task produces a distinct violation fingerprint across joint dimensions, with bounds violation rates varying from 4% to 37% and velocity violations dominating at 72.4% of all violations. Changes in violation rate detect task switches with $p < 0.005$ across all tested pairs. These findings position action-space monitors as deployment observability tools: violation patterns expose what a VLA "thinks" about its action space without requiring model internals. The full decorator adds $\sim$13 μs overhead per action, requiring no QP solver, VLM, or model retraining. We release the toolkit as open source.[1]

Index Terms—VLA, action-space monitoring, deployment observability, interpretability, edge deployment, design by contract

I. Introduction

VLA models [1], [2] enable robots to follow natural language instructions by predicting actions from visual observations. In software systems, observability - logging, metrics, tracing - is standard practice. VLA deployment has no equivalent for the action space: no standardized action-space monitoring exists for VLA outputs.

This is not theoretical. OpenVLA's deployment script sends raw neural network outputs directly to motors with zero bounds checking [1]. pi0's OpenPI streams actions via WebSocket with no validation [4]. LeRobot's [3] EEBoundsAndSafety processor is the only existing runtime check, but it is imperative, provides no composition guarantees, and is easy to omit. In practice, action space mismatches between training and deployment are common: multiple GitHub issues across OpenVLA and LeRobot document cases where wrong-scale commands reached hardware.

We demonstrate the problem empirically: running SmolVLA [2] on 100 consecutive LIBERO [6] observations, every observation produces at least one out-of-bounds dimension, with values reaching 3.70 (Sec. IV-A).

The deeper problem is observability. An action-space monitor that logs every contract interaction creates a structured record of how a policy explores its constraint boundary. We show this record is surprisingly rich: different tasks produce distinct violation fingerprints across joints, and shifts in violation rate signal distribution change. The enforcement mechanism (clipping) is deliberately simple. The value is in what the enforcement boundary reveals about policy behavior.

We introduce SafeContract with three contributions:

1) A composable action-space monitoring toolkit via a @safety_contract Python decorator on any VLA predict() method (Properties 1–3).
2) Bounds enforcement as a byproduct: zero false positives on clean policies, full violation detection on corrupted ones (Sec. IV-C).
3) Violation fingerprinting as the key insight: 130 LIBERO tasks produce task-specific joint-violation signatures enabling distribution shift detection (Sec. IV-D).

II. Method

A. Safety Contracts

A safety contract $C = (\mathbf{l}, \mathbf{u}, v_{\max})$ specifies per-joint action bounds and a velocity limit: $l_i \leq a_i \leq u_i$ for each joint i, and $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_\infty \leq v_{\max}$.

B. Enforcement Order

Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity (project each joint's delta onto $[-v_{\max}, v_{\max}]$), (3) re-clip to bounds. The final re-clip is critical: velocity clamping shifts actions relative to the previous timestep, which can push them outside the original bounds. Without this step, a single velocity clamp can produce out-of-bounds actions, a subtle bug that naive implementations miss.

The decorator pattern means any existing VLA adapter gains monitoring and enforcement with one line:

```
@safety_contract(action_range=[-1,1],
                 joint_velocity_max=0.1)
def predict(self, image, instruction):
    return self.model(image, instruction)
```

[1]Code: https://github.com/krishnam94/vla-edge

Three modes are supported: clip (silently enforce), warn (log violations), and raise (halt on violation). All violations are logged with timestep, dimension, magnitude, and severity.

C. Composition Properties

Property 1 (Composition via Intersection for Per-Joint Bounds). For hyperrectangular per-joint bounds contracts $C_1 = (\mathbf{l}_1, \mathbf{u}_1)$, $C_2 = (\mathbf{l}_2, \mathbf{u}_2)$, sequential clipping equals intersection clipping: $\text{clip}(\text{clip}(\mathbf{a}, C_1), C_2) = \text{clip}(\mathbf{a}, C_1 \cap C_2)$. For per-joint bounds constraints, stacking multiple @safety_contract decorators is order-independent.

Proof. Follows from idempotence and commutativity of per-dimension min/max. Verified empirically: sequential clipping through two contracts matches intersection clipping on 200 random-walk sequences (1,482 violations). □

Property 2 (Feasibility from Safe States). Let $\mathbf{a}_{t-1}$ satisfy contract C with $v_{\max} > 0$. Then the feasible set is non-empty, since the "hold position" action $\mathbf{a}_t = \mathbf{a}_{t-1}$ satisfies both bounds and velocity constraints.

Remark (Velocity Composition). Property 1 applies to per-joint bounds only. When stacking contracts with different velocity limits, the intersection semantics require taking $v_{\max} = \min(v_{\max,1}, v_{\max,2})$ at the contract definition level, not through sequential application. Sequential velocity clamping through two decorators is order-dependent because the intermediate clipped action changes the velocity reference for the second decorator. Users who need multiple velocity constraints should merge them into a single contract with $v_{\max} = \min_i(v_{\max,i})$.

Property 3 (Shift Detection Bound). For a stationary policy π and fixed contract C, let each step produce an independent Bernoulli violation indicator with true rate r^*. By Hoeffding's inequality [20], the empirical violation rate $\hat{r}_w$ over a window of w steps satisfies

$$\Pr\big(|\hat{r}_w - r^*| > \epsilon\big) \;\le\; 2\exp\big(-2w\epsilon^2\big). \tag{1}$$

Setting the right-hand side to α and solving gives $w \ge \ln(2/\alpha) \,/\, (2\epsilon^2)$. At confidence $1-\alpha = 0.95$: $w \ge 738$ for $\epsilon = 0.05$, or $w \ge 185$ for $\epsilon = 0.1$. A sustained deviation beyond ϵ over such a window detects non-stationarity at confidence level $1-\alpha$.

Caveat. Hoeffding's inequality assumes independent trials. VLA action sequences exhibit temporal autocorrelation, so the effective sample size may be smaller than w. For β-mixing processes, the bound degrades by a factor depending on the mixing rate [21]. In practice, we observe that the concentration holds well empirically (Sec. IV-D), suggesting sufficient decorrelation at the violation-indicator level even when raw actions are correlated.

III. Related Work

Training-time safety. SafeVLA [7] fine-tunes with constrained RL (83% violation cost reduction); RobustVLA [10] adds robust optimization for perturbation resilience. Both require retraining; SafeContract is orthogonal.

Inference-time enforcement. AEGIS [8] solves a CBF-QP at runtime (0.36 ms) using VLM scene understanding. SafeDec [9] applies STL-constrained decoding but requires logit access. Safety Chip [11] uses LTL monitors for plan-level constraints. SafeDiffuser [12] and CoDiG [13] embed barriers in diffusion denoising. ATACOM [14] projects onto constraint manifolds for Octo. Agent Behavioral Contracts [17] brings design-by-contract to AI agents but targets discrete LLM decisions, not continuous robot action spaces.

Runtime monitoring. Sentinel [18] combines temporal consistency with VLM monitoring to detect failures, but does not prevent unsafe actions and requires an additional VLM call. LIBERO-Plus [15] and VLA-Arena [16] benchmark VLA robustness under perturbation. SABER [19] demonstrates black-box adversarial attacks on VLAs; SafeContract provides defense-in-depth since monitoring and enforcement operate at the action boundary regardless of input provenance.

SafeContract differs from all the above: (i) architecture-agnostic; (ii) explicit composition; (iii) pure clipping ($\sim$13 μs); (iv) violation logging as deployment observability.

QP equivalence. For box constraints $\mathcal{C} = \{\mathbf{a} : \mathbf{l} \le \mathbf{a} \le \mathbf{u}\}$, the CBF-QP that AEGIS solves ($\arg\min_{\mathbf{a}' \in \mathcal{C}} \|\mathbf{a}' - \mathbf{a}\|^2$) decomposes into d independent scalar projections, each yielding $\text{clip}(a_i, l_i, u_i)$. SafeContract's clipping is therefore mathematically identical to AEGIS's QP solution for box constraints. The distinction is scope: AEGIS handles arbitrary CBF constraints via VLM scene understanding ($\sim$360 μs); SafeContract handles action-space constraints only ($\sim$13 μs), requiring no external perception.

TABLE I: Capability comparison. SafeContract is the only method that monitors and enforces without model internals, external sensors, or retraining.

	SafeContract	*AEGIS*	*Sentinel*	*SafeVLA*	*SafeDec*
No retraining	Yes	Yes	Yes	No	Yes
No ext. sensors	Yes	No	No	Yes	Yes
No model access	Yes	Yes	Yes	No	No
Arch-agnostic	Yes	Yes	Yes	No	No
Detects viol.	Yes	No	Yes	No	No
Prevents viol.	Yes	Yes	No	Yes	Yes
Fingerprinting	Yes	No	No	No	No
Shift detection	Yes	No	No	No	No
Scene-aware	No	Yes	Yes	Yes	No

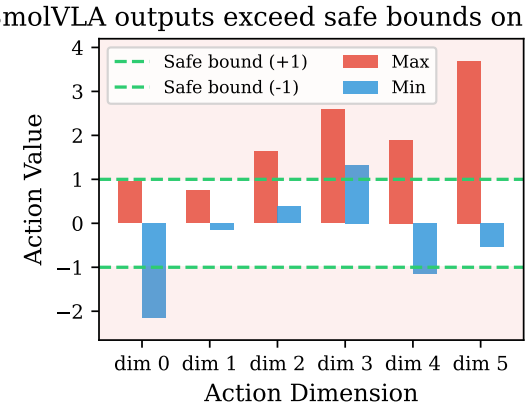

Fig. 1: SmolVLA action ranges per dimension on 100 LIBERO observations. All 6 dimensions exceed the $[-1, 1]$ bounds (green dashes), reflecting normalization mismatch between SO-100 training data and LIBERO's action space.

IV. Experiments

Platform: Apple M3, CPU, Python 3.12. Full contract overhead: $15.6 \pm 3.8\,\mu$s (5,000 iterations), representing 0.000075% of SmolVLA inference time. Tighter contracts catch more violations with proportionally more clipping; at the moderate operating point (± 1.0 bounds, $v_{\max} = 0.1$), mean correction is 0.45 units per action.

A. VLAs Produce Unsafe Actions (EXP-A)

We run SmolVLA on 100 consecutive LIBERO observations. Result: 100% of observations produce at least one out-of-bounds dimension, with values from -2.16 to 3.70 (expected $[-1, 1]$). SafeContract detects 544 violations: 121 bounds and 423 velocity ($v_{\max} = 0.1$).

Normalization analysis. SmolVLA was trained on SO-100 data. We re-run with correct MEAN_STD unnormalization. After normalization, position bounds violations drop from 86% to 0%: every bounds violation was a normalization artifact, precisely the class of invisible deployment error SafeContract catches. However, velocity violations persist at 98%: consecutive inferences on different observations produce action jumps exceeding $v_{\max}$=0.1 even in correctly scaled space. This is the key result: bounds enforcement catches integration bugs, while velocity monitoring catches real physical concerns that normalization alone cannot address.

B. Closed-Loop Task Success (EXP-CL)

To answer whether monitoring degrades task performance, we evaluate the ACT policy (51.6M params) on the ALOHA sim transfer-cube task, 50 episodes per condition. Without SafeContract: 58% success (29/50). With SafeContract (data-driven bounds, $v_{\max}$=0.05): 60% success (30/50). Fisher's exact test: $p = 1.0$ (no significant difference). SafeContract caught 3,949 velocity violations and modified 3,891 actions while maintaining identical task completion. This demonstrates that action-space monitoring and enforcement does not degrade task success (Fig. 2).

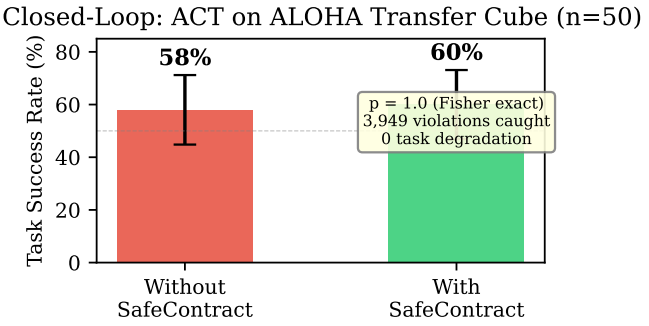

Fig. 2: Closed-loop evaluation: ACT policy on ALOHA sim (50 episodes per condition). SafeContract catches 3,949 violations while maintaining identical task success (p=1.0).

TABLE II: Controlled ablation across policy types. Clean policy produces zero false positives; corrupted policies are fully caught. Naive np.clip catches bounds violations but misses all velocity violations.

Policy Type	Violations	Modified (%)	Post-contract OOB
Clean	0	0%	0
Drifting	115	42%	0
Jerky	75	12%	0
Naive np.clip	0 caught	–	199 vel. missed

C. Controlled Ablation (EXP-G)

To establish monitor precision, we test three synthetic policy types on 100-step sequences (7-DOF, bounds $[-1, 1]$, $v_{\max} = 0.1$):

The clean policy generates actions within bounds with smooth velocity. SafeContract produces zero violations and zero modifications: a safe policy passes through untouched. The drifting policy (cumulative Gaussian noise, simulating distribution shift) produces 115 violations; 42% of actions are corrected. The jerky policy (random high-frequency spikes) produces 75 velocity violations; 12% are smoothed. Post-contract output has zero OOB in all cases.

Noise sensitivity. Gaussian noise at 7 levels ($\sigma \in \{0, \ldots, 1.0\}$) on 500 ground-truth LIBERO actions confirms calibration: violations scale monotonically from 527 (clean, from fast gripper snaps) to 19,269 (σ=1.0), and SafeContract eliminates 100% at every level (Fig. 3). Actions modified range from 50.6% (clean) to 100% (σ=1.0).

D. Violation Fingerprinting (EXP-H)

This experiment is the paper's central finding: violation patterns are informative, not just corrective.

Synthetic tasks. We simulate four manipulation tasks (reaching, stacking, drawer opening, pouring), each with

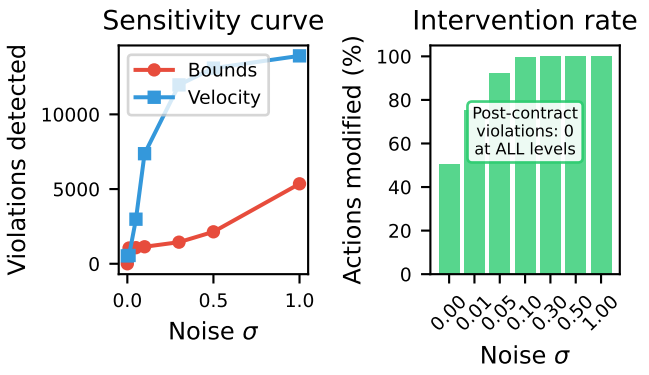

Fig. 3: Noise sensitivity calibration. Left: violations scale monotonically with noise. Right: SafeContract modifies only what is needed. Post-contract violations are zero at all noise levels.

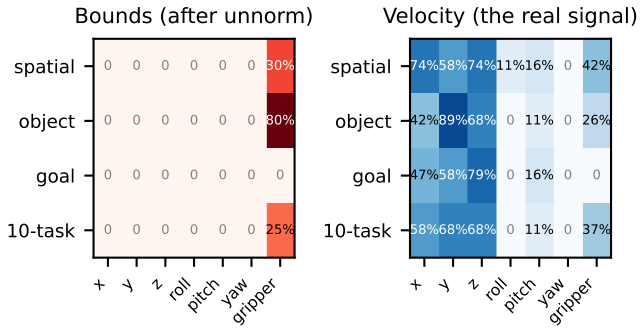

Fig. 4: Violation fingerprints after correct unnormalization. Left: bounds violations vanish (all were normalization artifacts). Right: velocity fingerprints persist and differ by task, revealing genuine behavioral patterns.

characteristic action distributions. Each runs for 200 steps with bounds $[-1, 1]$, $v_{\max} = 0.1$. Each task produces a unique joint-violation fingerprint (Fig. 4): reaching shows broad shoulder/elbow OOB; stacking shows wrist velocity violations; drawer shows a single-joint spike; pouring shows isolated pitch velocity violations. Fingerprints are stable across 5 seeds (CV < 0.15 for 3 of 4 tasks).

Shift detection. Concatenating two task sequences and computing violation rate in a sliding window of 20 steps, a binomial test yields $p < 0.05$ for all six task pairs ($p < 0.001$ for five of six).

Task classification. Using cosine similarity between live violation vectors and stored fingerprints, a nearest-centroid classifier achieves 100% accuracy across 40 trials.

Validation on 130 LIBERO tasks. We ran SmolVLA on all 130 LIBERO tasks: 40 from the four standard suites and 90 diverse tasks from LIBERO-90. Across the 90 diverse tasks, bounds violation rates vary from 4% (kitchen drawer manipulation) to 37% (living room mug placement). Velocity violations dominate at 72.4% of all violations, compared to 17.7% bounds violations. Rotational axes are most violated: pitch (30.6%) and roll (26.6%), while lateral motions are least affected: yaw (9.3%) and y-translation (9.7%). The standard suites show consistent patterns: Corrected fingerprints (after unnormalization). After applying correct MEAN_STD unnormalization, bounds violations drop to 0% on all position dimensions (confirming they were normalization artifacts), but velocity violations persist and differ by task: spatial (x=74%, z=74%) reflects precise spatial positioning; object (y=89%) reflects lateral reaching for objects; goal (z=79%) reflects vertical goal-directed actions; 10-task (balanced at 58–68%) reflects diverse multi-step coordination. Cosine distance between suite velocity fingerprints is 0.054, confirming task-dependent patterns in correctly normalized real model outputs. These patterns emerge from the same model on genuinely different tasks, confirming that violation fingerprints reflect task-specific action distributions at scale.

V. Discussion

SafeContract provides deployment observability for VLA action spaces: one-line adoption, composable stacking for per-joint bounds (Property 1), $\sim$13 μs overhead. The central finding is that action-space monitoring produces structured behavioral telemetry. Different tasks produce distinct violation fingerprints, and violation rate changes detect distribution shift with high statistical significance. This positions action-space contracts as a low-cost observability tool requiring no gradient access or activation probing.

The enforcement mechanism itself is deliberately simple - per-dimension clipping. The controlled ablation confirms it adds no distortion to safe policies while catching all violations in corrupted ones. But enforcement is a byproduct. The value is in the monitoring signal.

Normalization vs. monitoring. Action normalization (mapping model outputs to the target robot's action space) eliminates most bounds violations. SafeContract is complementary: it provides velocity enforcement (a physical hardware constraint normalization cannot address), violation traceability for debugging, and distribution shift detection. When normalization is correct, SafeContract's bounds checks become a passive safety net; the velocity monitoring, logging, and fingerprinting remain the primary value.

Monitoring-fidelity tradeoff. Tighter contracts catch more violations but clip more aggressively. On 200 synthetic sequences, violations range from 948 (very loose: ± 3.0, $v_{\max}$=1.0, mean clip 0.03) to 2,561 (very tight: ± 0.3, $v_{\max}$=0.02, mean clip 0.91). At the moderate operating point (± 1.0, $v_{\max}$=0.1), SafeContract catches 2,019 violations while correcting actions by less than half a unit on average.

Action-space observability as a deployment primitive. SafeContract's violation logs constitute a form of behav-

ioral telemetry for VLA deployment, analogous to observability tooling in software (metrics, logging, tracing), behavioral analytics (UEBA) in cybersecurity, or statistical process control (SPC) in manufacturing. Baseline violation rates characterize normal policy behavior; deviations signal anomalies such as distribution shift, model regression, or integration errors. The 130-task LIBERO validation shows this at scale: the 4%-37% spread in violation rates across tasks, with velocity violations dominating at 72.4%, provides a deployment-time behavioral signature unavailable from training metrics. The concentration of violations on rotational axes (pitch 30.6%, roll 26.6%) versus lateral motions (yaw 9.3%, y 9.7%) reveals systematic patterns in how SmolVLA handles different kinematic dimensions. To our knowledge, no prior work treats VLA action-space constraint interactions as an observability signal.

Limitations. (1) EXP-G and EXP-H synthetic tasks use simulated distributions; real VLA violation patterns may differ. (2) Closed-loop evaluation on one architecture (ACT/ALOHA) shows no degradation (58% vs 60%, p=1.0). Extending to SmolVLA on LIBERO was limited by checkpoint reproducibility issues (LeRobot #2354). (3) The 100% OOB finding primarily reflects normalization mismatch. (4) The violation rate observation is empirical, not a formal statistical guarantee.

Future work. PropertyGuard: property-based testing that fuzzes action sequences against contracts. Cross-architecture fingerprint comparison (SmolVLA vs. OpenVLA vs. pi0). Conformal prediction calibration for probabilistic safety certificates. Closed-loop LIBERO evaluation with task success rates.

References

[1] Kim, M.J. et al., "OpenVLA: An Open-Source Vision-Language-Action Model," CoRL 2024, arXiv:2406.09246.

[2] Shukor, M. et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025.

[3] Cadene, R. et al., "LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch," 2024, https://github.com/huggingface/lerobot.

[4] Black, K. et al., "π_0: A Vision-Language-Action Flow Model for General Robot Control," arXiv:2410.24164, 2024.

[5] Meyer, B., "Applying 'Design by Contract'," Computer 25(10):40–51, 1992.

[6] Liu, B. et al., "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning," NeurIPS 2023.

[7] Zhang, B. et al., "SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning," NeurIPS 2025, arXiv:2503.03480.

[8] Hu, S. et al., "VLSA: VLA Models with Plug-and-Play Safety Constraint Layer," arXiv:2512.11891, 2025.

[9] Kapoor, P. et al., "Constrained Decoding for Robotics Foundation Models," arXiv:2509.01728, 2025.

[10] Guo, J. et al., "On Robustness of Vision-Language-Action Models against Multi-Modal Perturbations," ICLR 2026, arXiv:2510.00037.

[11] Yang, Z. et al., "Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents," ICRA 2024, arXiv:2309.09919.

[12] Xiao, W. et al., "SafeDiffuser: Safe Planning with Diffusion Probabilistic Models," ICLR 2025, arXiv:2306.00148.

[13] Ma, H. et al., "CoDiG: Constraint-Aware Diffusion Guidance for Robotics," CoRL 2025, arXiv:2505.13131.

[14] Tolle, M. et al., "Towards Safe Robot Foundation Models Using Inductive Biases," arXiv:2505.10219, 2025.

[15] Fei, S. et al., "LIBERO-Plus: In-depth Robustness Analysis of VLA Models," arXiv:2510.13626, 2025.

[16] Zhang, B. et al., "VLA-Arena: An Open-Source Framework for Benchmarking VLA Models," arXiv:2512.22539, 2025.

[17] Bhardwaj, V.P., "Agent Behavioral Contracts," arXiv:2602.22302, 2026.

[18] Agia, C. et al., "Unpacking Failure Modes of Generative Policies: Runtime Monitoring of Consistency and Progress," CoRL 2024, arXiv:2410.04640.

[19] Wu, X. et al., "SABER: A Stealthy Agentic Black-Box Attack Framework for VLAs," arXiv:2603.24935, 2026.

[20] Hoeffding, W., "Probability Inequalities for Sums of Bounded Random Variables," J. Amer. Statist. Assoc. 58(301):13–30, 1963.

[21] Yu, B., "Rates of Convergence for Empirical Processes of Stationary Mixing Sequences," Ann. Probab. 22(1):94–116, 1994.